Hooks

Introduction

- Added to React in version 16.8
- Let to use state and other React features in function components
- Types of hooks:
  - State hooks
  - Effect hooks
  - Context hooks
  - Memoized hooks
  - Callback hooks
  - Reference hooks
  - Reducer hooks
  - …

State Hooks

- Allow to declare states in function components
- Use `useState()` for setting and retrieving a state

- Example:

```
function Regulator() {
  const [value, setValue] = useState(0);

  return <>
    <Button title="Decrease"
      onPress={() => setValue(value - 1)} />
    <Text>Volume: {value}</Text>
    <Button title="Increase"
      onPress={() => setValue(value + 1)} />
  </>
}
```

Multipe State Variables vs One Complex Object

- Multiple state variables:
  - ```
    const [id, setId] = useState(-1);
    const [username, setUsername] = useState("");
    const [email, setEmail] = useState("");
    ```
  - Better in most cases
- One object state variable:
  - ```
    const [userInfo, setUserInfo] = useState({
      id: -1,
      username: "",
      email: ""
    });
    ```
  - Remember to copy all the other fields along with the specific fields that need to be updated
  - Cloning deeply nested objects can be expensive

Functional State Updaters

- State updates are not immediately affective, but batched
- Problem: multiple calls may result in incorrect behavior
  - ```
    const increaseValue =
            () => setValue(value + 1);
    increaseValue();
    increaseValue();
    increaseValue();
    ```
  - `value` is increased by 1, not 3 as expected

- Solution: use functional state updater
  - ```
    const increaseValue =
            () => setValue(v => v + 1);
    increaseValue();
    increaseValue();
    increaseValue();
    ```

Exercise

- Create an app that allows to switch between light and dark themes by a `<Switch>`

Effect Hooks

- Allow to perform side effects in function components
- Use `useEffect(func, dependency)`

- Example: save value to storage when updated
  - ```
    function Regulator({valueName}) {
      const [value, setValue] = useState(0);

      useEffect(() => {
        storeValue(valueName, value);
      });

      return <>...</>
    }
    ```
    ```

Running on Every Render

- No dependency (2nd parameter) given:
  - `useEffect(() => {…});`

- Example: see last example

Running Only on the First Render

- Pass an empty array as dependency:
  - `useEffect(() => {…}, []);`

- Example:
  - ```
    useEffect(() => {
      setInterval(() => {
        //...
      }, 1000);
    }, []);
    ```
  - Question: What would happen if the hook runs on every render?
    ```

Running when a Dependency Changes

- Also runs on the first render
- Pass a combination of props and/or states as dependency:
  - `useEffect(() => {…}, [prop1, prop2, state1, state2]);`
  - Hook runs on the 1st render, and when any value of the dependency changes
- Example:
  - ```
    function Multiplier() {
      const [x, setX] = useState(1);
      const [y, setY] = useState(2);
      const [result, setResult] = useState(0);

      useEffect(() => setResult(x * y), [x, y]);

      return <>…</>;
    }
    ```
    ```

Effect Cleanup

- Some effects require cleanup to reduce resource leaks
  - Ex: timeouts, subscriptions, event listeners
- Return a function at the end of `useEffect` hook, which will get executed when the component unmounts and before executing the next effect

- Example:
  - ```
    useEffect(() => {
      const timer = setTimeout(() => {…}, 1000);
      return () => clearTimeout(timer);
    }, []);
    ```
    ```

Exercise

- Create an app that evaluates the following two equations:
  - $t_1 = x + y \times r_1$
  - $t_2 = x + z \times r_2$

  where:
  - x, y, z are user-specified values
  - r_1, r_2 are random values which change every 3s and 5s, respectively

For Class Components

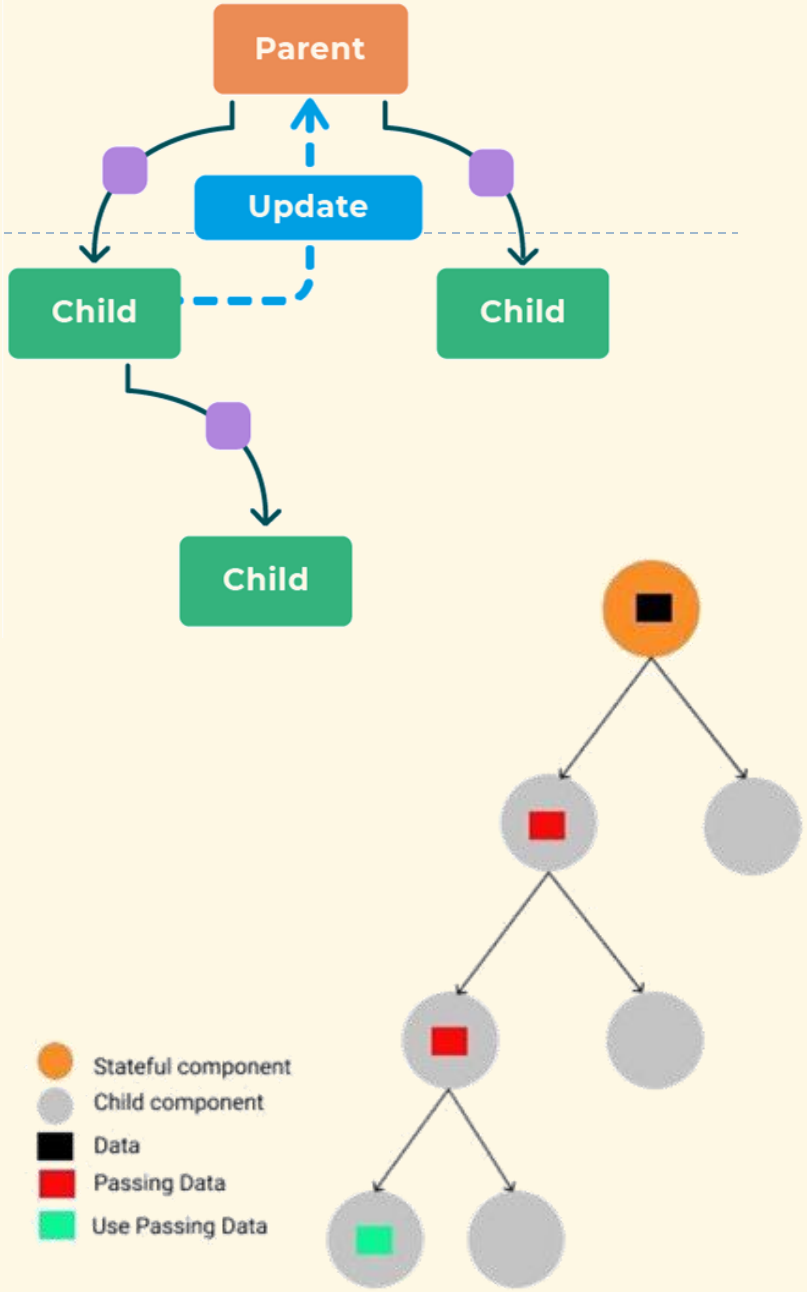
- Use the following handler methods:
  - `componentDidMount()`: after the component is mounted
  - `componentDidUpdate(prevProps, prevState)`: after updating occurs
  - `componentWillUnmount()`: before the component is unmounted

Exercise

- Do the last exercise using class component

Context Hooks

- State should be held by the highest parent component in the stack that requires access to the state
- Prop drilling problem: even components in the middle do not need the state, they must pass it along to children needing it
- Use `createContext()` to solve the problem
  - Context provider: parent component that supplies the state
  - Context consumer: child component that uses the state



Example

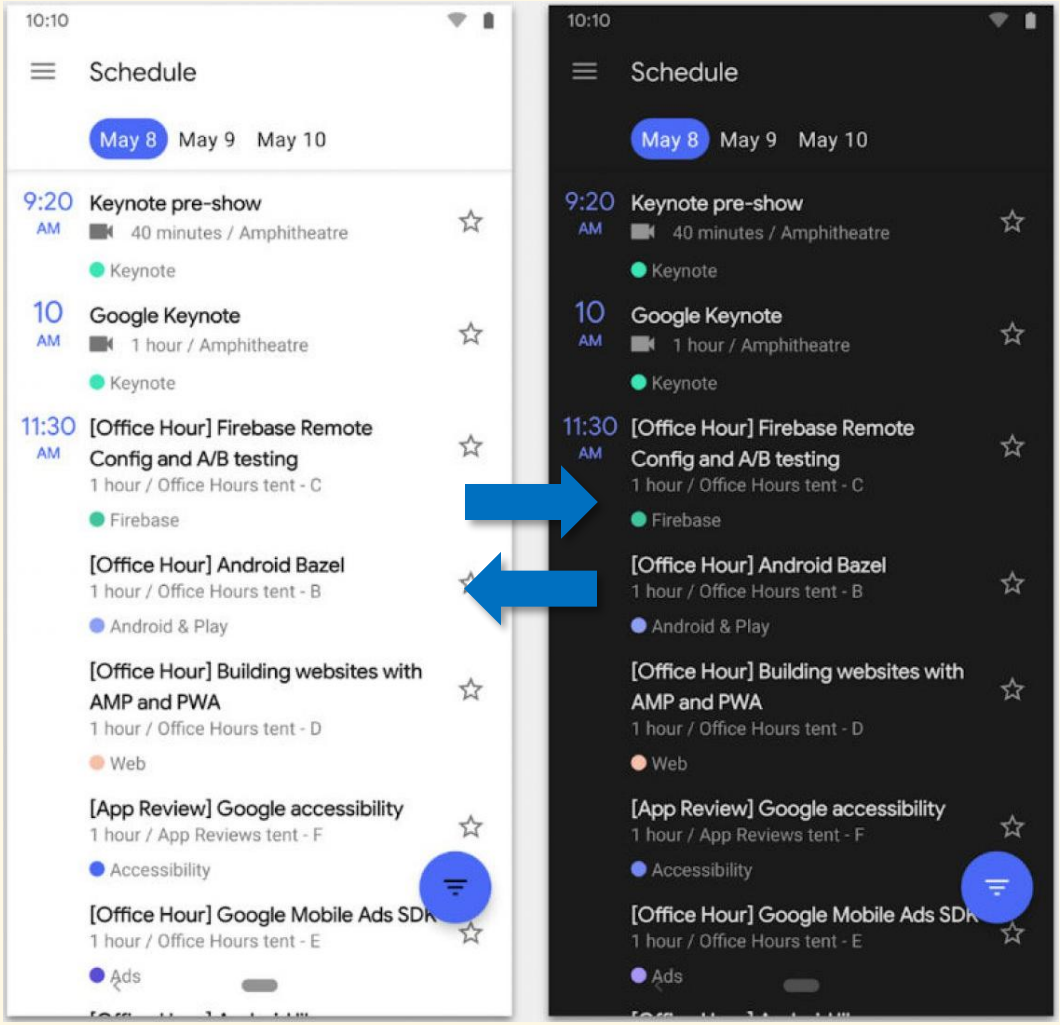
```
const XContext = createContext();

function Controller() {
  const {x, setX} = useContext(XContext);
  return <>
    <Text>X: {x}</Text>
    <Button title="-" onPress={() => setX(x-1)} />
    <Button title="+" onPress={() => setX(x+1)} />
  </>
}

function App() {
  const [x, setX] = useState(1);
  return <XContext.Provider value={{x, setX}}>
    <View>
      <Controller />
      <Text>X squared: {x * x}</Text>
    </View>
  </XContext.Provider>
}
```

Exercise

- Modify the theme-switching app to use context
  - The app should contain several child components whose appearance is updated according to the theme



Memoized Hooks

- Memoization: caching a value so that it does not need to be recalculated
  - Performance optimization purpose
- Counter example: `intCal` function is called in every render
  - ```
    function intCal(x) { // intensive calculation
      return x + 5;
    }

    function MyComponent() {
      const [num, setNum] = useState(0);
      const [light, setLight] = useState(true);
      const result = intCal(num);
      return ...
    }
    ```
    ```

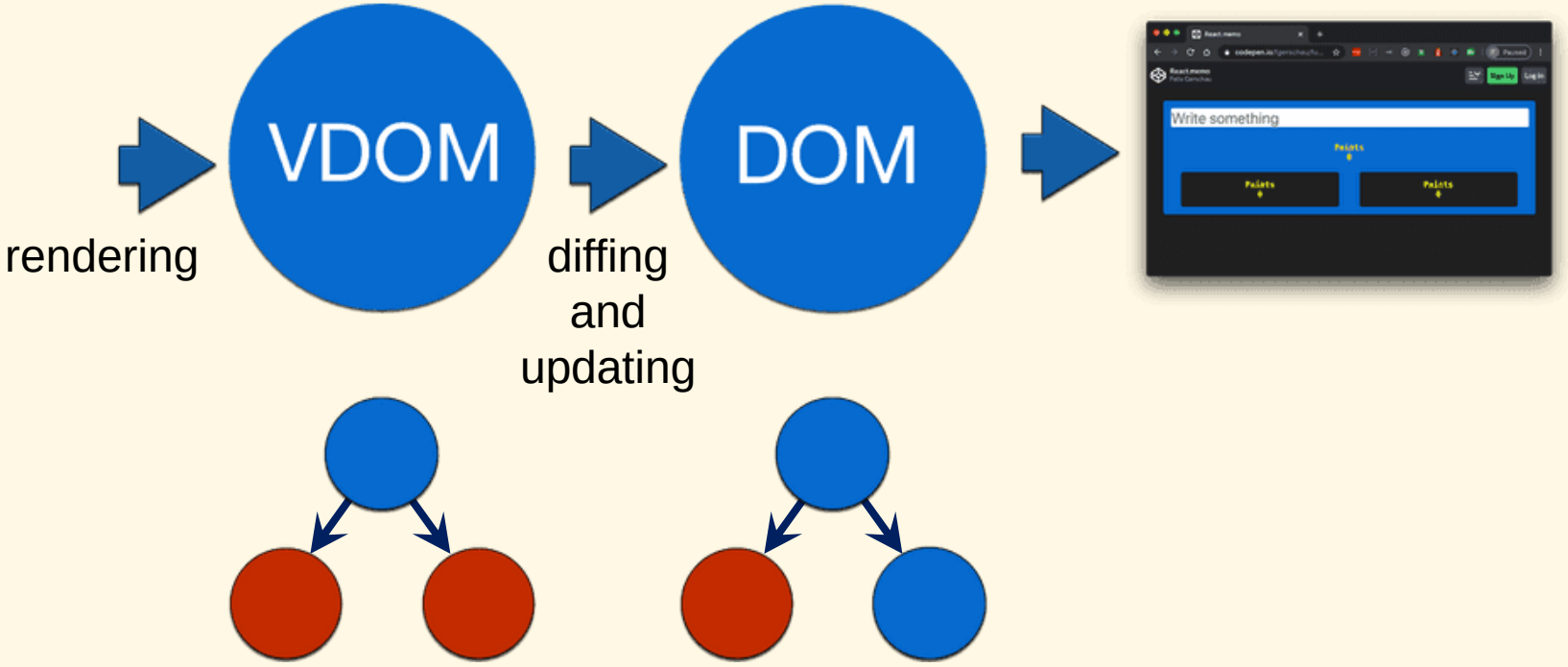
useMemo

- Use cases: to avoid re-execution of an intensive function (e.g., sort, calculation, data request,…) when its dependency is not updated
- Example: Make `intCal` to get executed only when `num` is changed
  - ```
    function MyComponent() {
      const [num, setNum] = useState(0);
      const [light, setLight] = useState(true);

      const result =
        useMemo(() => intCal(num), [num]);

      return ...
    }
    ```
    ```

React Rendering Revisited



- Component rendering is triggered when its props or states change
  - When a component re-renders, all its children also do recursively
  - Updating the VDOM doesn't necessarily trigger an update of the real DOM

memo

- The default render mechanism is fine for simple components
- But if the rendering is expensive, use `memo(…)` to skip re-rendering a component when its props are not changed
  - Compared with `Object.is()`, or a custom comparison function if is given
  - Only props are compared, states and context are not
- Examples:
  - ```
    const ShippingForm = memo(
      function SlowShippingForm({ onSubmit }) {
        // ...
      }
    );
    ```
  - ```
    const ShippingForm = memo(({ onSubmit }) => {
      // ...
    });
    ```
  - ```
    const Chart = memo(({ dataPoints }) => {
      // ...
    }, (oldProps, newProps) => {
      // custom comparison function...
    });
    ```
    ```

Callback Hooks

- Problem: An inline function created within a function component may cause unnecessary re-rendering (as the function identity changes on every render)
- Example:

```
function Controller() {
  const [x, setX] = useState(0);
  const decrease = () => setX(x - 1);
  const increase = () => setX(x + 1);

  return <>
    <Text>X: {x}</Text>
    <Button title="-" onPress={decrease} />
    <Button title="+" onPress={increase} />
  </>
}
```

useCallback

- Allows to avoid re-creating an inline function on re-rendering
  - ```
    function Controller() {
      const {x, setX} = useState(0);
      const decrease =
              useCallback(() => setX(x - 1), [x]);
      const increase =
              useCallback(() => setX(x + 1), [x]);

      return <>
        <Text>X: {x}</Text>
        <Button title="-" onPress={decrease} />
        <Button title="+" onPress={increase} />
      </>
    }
    ```
  - Returns the same function if the dependency is not changed
    - `useMemo(…)` returns a value, while `useCallback(…)` returns a function
  - `memo(…)` may need to be used together with `useCallback(…)`
  - Would be more useful when the optimized components are big
    ```

Exercise

- Add debugging logs to previous examples and check the difference when using and not using `useMemo`, `useCallback` and `memo`

Reference Hooks

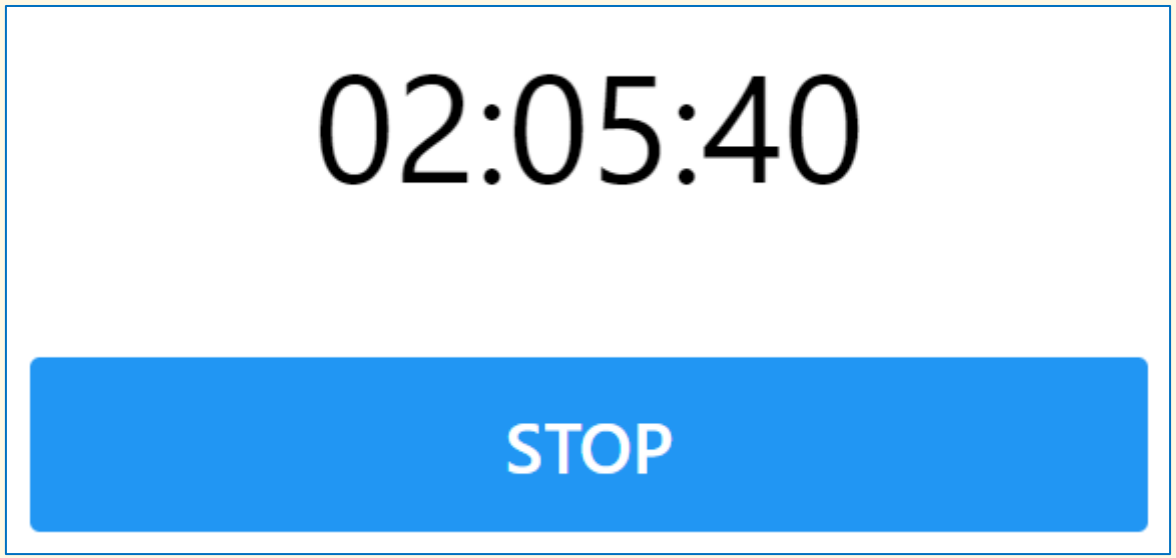
- Used for mutable values, which don't cause re-rendering when being changed
- Example: logging button clicks without re-rendering
  - ```
    function LogButtonClicks() {
      const countRef = useRef(0);

      const handler = () => {
        countRef.current++;
        console.log(`Clicks: ${countRef.current}`);
      };

      console.log("render");
      return <button onClick={handler}>Click</button>;
    }
    ```
- Note that `useRef` is also useful to hold a reference to DOM HTML elements when working with the web
    ```

Exercise

- Create a stopwatch: a counter increasing every second, and a button to stop/resume it
  - Hint: use `useRef` to keep a reference to the timer handle



Reducer Hooks

- To be visited later

Final Notes

- Only call hooks at the top level
  - Don’t call them inside loops, conditions, or nested functions
- Only call hooks from React functions
  - Don’t call hooks from regular JavaScript functions